Tool 정의 심화 (실습)

1-1. 좋은 도구의 조건
1-2. @tool 데코레이터와 스키마 (실습)
1-3. 도구 에러 처리 (실습)

1-1. 좋은 도구의 조건

> 도구의 이름, 설명, 인자 스키마가 도구 선택 정확도를 좌우한다는 원리를 이해해봅니다.

3일차에서 Agent의 뼈대인 ReAct 루프를 만들었습니다. 오늘은 그 Agent가 실제로 일을 하게 만드는 도구(Tool)를 깊게 다룹니다.

LLM은 도구의 코드를 보지 못합니다. LLM이 보는 것은 딱 세 가지입니다.

1. 도구 이름 (예: `search_employee`)
2. 도구 설명 (함수의 docstring으로 함수 바로 아래에 적는 설명 문자열)
3. 인자 스키마 (인자 이름, 타입, 각 인자의 설명)

모델은 이 세 가지 텍스트만 보고 지금 이 도구를 부를지 어떤 인자로 부를지를 결정합니다. 그래서 도구 정의는 코드 작성이라기보다 **LLM에게 보내는 프롬프트 작성**에 가깝습니다. docstring은 사람이 읽는 주석이 아니라 모델이 읽는 사용 설명서입니다.

좋은 도구 설명에 꼭 들어가야 할 항목은 다음과 같습니다.

항목	예시
무엇을 하는가	"임직원 디렉터리를 조회한다"
언제 쓰는가	"직원의 이메일, 소속 팀이 필요할 때"
언제 쓰지 말아야 하는가	"사내 규정 질문은 문서 검색 도구를 사용"
인자 설명	"정확한 이름. 부분 일치는 지원하지 않음"
실패 시 무엇이 돌아오는가	"없으면 '찾을 수 없습니다'를 반환"

나쁜 예와 좋은 예를 비교해봅니다.

```
# 나쁜 예: 모델이 언제 써야 할지 알 수 없음
@tool
def search_employee(name: str) -> str:
    """직원 검색"""
    ...

# 좋은 예: 언제 쓰고 무엇이 돌아오는지까지 명확함
@tool
def search_employee(name: str) -> str:
    """임직원 디렉터리에서 이름으로 직원 정보를 조회한다.

    직원의 이메일, 소속 팀이 필요할 때 사용한다.
    사내 규정이나 정책 질문에는 사용하지 않는다 (문서 검색 도구 사용).

    Args:
        name: 직원의 정확한 한국어 이름. 부분 일치는 지원하지 않는다.

    Returns:
        "이름 / 팀 / 이메일" 형식 문자열. 없으면 "찾을 수 없습니다" 반환.
    """
    ...
```

도구를 여러 개 등록했는데 모델이 엉뚱한 도구를 고를 때가 있습니다. 대부분은 코드 버그가 아니라 설명이 서로 겹치거나 모호한 것이 원인입니다. "언제 쓰지 말아야 하는가"를 설명에 추가하면 눈에 띄게 좋아집니다.

한국어 질문을 받는 서비스라면 docstring도 한국어로 쓰는 편이 선택 정확도에 유리합니다.

> 도구 선택 정확도는 코드가 아니라 이름, 설명, 스키마라는 텍스트가 좌우한다는 걸 확인했습니다.

1-2. @tool 데코레이터와 스키마 (실습)

> 함수를 도구 객체로 바꾸고 Pydantic으로 인자를 검증해봅니다.

`@tool` 은 평범한 파이썬 함수를 LLM이 호출할 수 있는 도구 객체로 포장하는 데코레이터입니다. **파이썬의 데코레이터는 Java의 어노테이션에 대응하는 개념**입니다.
스프링에서 `@RestController` 를 붙이면 평범한 클래스가 HTTP 요청을 받는 컨트롤러로 바뀝니다.
`@tool` 을 붙이면 평범한 함수가 LLM 호출 가능한 도구로 바뀝니다.

둘 사이에 차이점이 하나 있습니다. Java 어노테이션은 메타데이터 표시일 뿐이고 실제 처리는 프레임워크가 리플렉션으로 합니다. 반면 파이썬 데코레이터는 그 자체가 "함수를 받아 다른 객체를 돌려주는 함수"입니다.

`@tool` 을 통과한 `add` 는 더 이상 함수가 아니라 `StructuredTool` 객체입니다. 그래서 `add(3, 5)` 처럼 직접 부르면 안 되고 `add.invoke({"a": 3, "b": 5})` 처럼 불러야 합니다. 처음에 가장 많이 틀리는 부분입니다.

따라하기

1. 오늘 실습 폴더를 만들고 가상환경을 활성화한 뒤 패키지를 설치합니다. 자격 증명은 레포 루트의 `.env` 를 그대로 쓰므로 복사할 필요 없습니다.

```
mkdir day4_practice
cd day4_practice
..\.venv\Scripts\activate
# 오늘 쓰는 mcp, fastmcp, langchain-mcp-adapters는 1일차 requirements.txt에 들어 있습니다 (추가 설치 없음)
```

2. `@tool`을 세 단계 깊이로 써 보는 스크립트를 만듭니다. 아래 코드를 `day4_practice/tools_basic.py`로 저장합니다.

```python
# tools_basic.py - @tool 데코레이터 3단계 깊이
from langchain_core.tools import tool
from pydantic import BaseModel, Field


# 단계 1: 가장 단순한 형태
# 이름은 함수 이름, 설명은 docstring, 스키마는 타입 힌트에서 자동 추론
@tool
def add(a: int, b: int) -> int:
    """두 정수를 더한다."""
    return a + b


# 단계 2: 이름과 설명을 데코레이터 인자로 명시
# 도구 이름은 calculator, 함수 이름은 _internal_calc로 분리 가능
@tool("calculator", description="수학 계산을 정확하게 수행한다. LLM은 큰 숫자 계산에 약하므로 계산이 필요하면 이 도구를 쓴다.")
def _internal_calc(expression: str) -> str:
    return str(eval(expression, {"__builtins__": {}}, {}))


# 단계 3: Pydantic으로 인자 스키마를 강제
# 타입 힌트만으로 부족할 때 (검증 규칙, 기본값, 인자별 설명)
class SearchEmployeeInput(BaseModel):
    name: str = Field(
        description="직원의 정확한 한국어 이름 (부분 일치 불가)",
        min_length=2, max_length=20,   # 너무 짧거나 긴 값을 실행 전에 차단
    )


@tool(args_schema=SearchEmployeeInput)
def search_employee(name: str) -> str:
    """삼성SDS 임직원 디렉터리에서 이름으로 직원 정보를 조회한다."""
    # 교육용 더미 데이터입니다
    fake_db = {
        "김하늘": "김하늘 / 클라우드운영팀 / 팀장 / haneul.kim@samsungsds.example.com",
        "박도윤": "박도윤 / 물류플랫폼팀 / 백엔드 / doyun.park@samsungsds.example.com",
    }
    return fake_db.get(name, f"'{name}' 직원을 찾을 수 없습니다.")


if __name__ == "__main__":
    # 도구는 함수가 아니라 객체이므로 .invoke()로 호출합니다
    print(add.invoke({"a": 3, "b": 5}))
    print(_internal_calc.invoke({"expression": "15 * 25"}))
```

```
    print(search_employee.invoke({"name": "김하늘"}))

    # Pydantic 검증: 잘못된 인자는 도구 실행 전에 차단됩니다
    try:
        search_employee.invoke({"name": "김"})   # min_length=2 위반
    except Exception as e:
        print(f"검증 에러 (예상된 동작): {type(e).__name__}")

    print("=== 도구 메타데이터 (LLM이 실제로 보는 것) ===")
    for t in [add, _internal_calc, search_employee]:
        print(f"이름: {t.name}")
        print(f"설명: {t.description}")
        print(f"스키마: {t.args_schema.model_json_schema() if t.args_schema el
se None}")
        print()
```

3. 실행해서 세 도구의 호출 결과와 마지막의 메타데이터 출력을 확인합니다. 이름, 설명, 스키마가 1-1에서 말한 "LLM이 보는 전부"입니다.

```
python tools_basic.py
```

막힐 때

- `add(3, 5)` 처럼 직접 호출하면 `TypeError` 가 납니다.

  `@tool` 을 거친 순간 함수가 아니라 도구 객체입니다. `add.invoke({"a": 3, "b": 5})` 처럼 dict로 호출합니다.

- docstring이 없는 함수에 `@tool` 을 붙이면 에러가 납니다.

  설명이 없는 도구는 LLM에게 의미가 없기 때문에 일부러 막아 둔 것입니다. 한 줄이라도 설명을 답니다. 다만 위 2단계처럼 데코레이터에 `description` 을 직접 넘기면 docstring이 없어도 됩니다. 둘 중 하나는 반드시 있어야 한다고 기억하면 됩니다.

- `@tool()` 과 `@tool` 둘 다 되지만 인자를 줄 때는 반드시 괄호를 씁니다 (`@tool("calculator", ...)`).

확인: 세 도구가 invoke로 실행되고 `"김"` 입력에서 ValidationError가 나며 메타데이터가 출력되면 성공입니다.

> 데코레이터가 함수를 도구 객체로 바꾸고 Pydantic이 잘못된 인자를 실행 전에 차단한다는 걸 확인했습니다.

1-3. 도구 에러 처리 (실습)

> 도구 실패를 예외 대신 복구 지침이 담긴 메시지로 돌려줘 Agent가 스스로 복구하게 만들어봅니다.

도구가 실패하는 순간 선택지는 두 가지입니다.

1. 예외를 그대로 터뜨린다: 그래프 전체가 중단됩니다. 사용자에게는 500 에러 같은 경험입니다.
2. 실패를 **자연어 메시지로 돌려준다**: LLM이 그 메시지를 읽고 인자를 고쳐 재시도하거나 다른 도구를 쓰거나 사용자에게 상황을 설명합니다.

운영에서는 2번이 기본입니다. try/except로 예외를 잡고, 잡은 뒤에는 다시 던지지 않고 "다음 행동 지침이 담긴 문자열"을 반환합니다.

에러 메시지가 곧 LLM에게 주는 복구 안내문이 됩니다.

따라하기

1. 실패를 메시지로 돌려주는 도구를 만듭니다. 아래 코드를 `day4_practice/tool_error.py` 로 저장합니다.

```
# tool_error.py — 실패를 메시지로 돌려주고 Agent가 복구하게 만들기
from langchain_core.tools import tool

# 교육용 더미 데이터입니다
EMPLOYEES = [
    {"name": "김하늘", "team": "클라우드운영팀", "email": "haneul.kim@samsungsds.example.com"},
    {"name": "박도윤", "team": "물류플랫폼팀", "email": "doyun.park@samsungsds.example.com"},
    {"name": "이서준", "team": "클라우드운영팀", "email": "seojun.lee@samsungsds.example.com"},
]


@tool
def search_employee(name: str) -> str:
    """삼성SDS 임직원 디렉터리에서 이름으로 직원 정보를 조회한다.

    Args:
        name: 직원의 정확한 한국어 이름 (최소 2자)

    Returns:
        직원 정보 문자열, 또는 다음 행동을 안내하는 에러 메시지
    """
    try:
        # 1) 입력 검증 실패 -> raise 대신 안내 메시지 반환
        if not name or len(name) < 2:
            return "에러: 이름이 너무 짧습니다. 최소 2자 이상의 정확한 한국어 이름으로 다시 호출하세요."

        # 2) 정상 검색
        for emp in EMPLOYEES:
            if emp["name"] == name:
                return f"{emp['name']} / {emp['team']} / {emp['email']}"

        # 3) 결과 없음 -> 후보를 함께 알려주면 LLM이 다음 판단을 하기 쉬움
        candidates = ", ".join(e["name"] for e in EMPLOYEES)
        return f"'{name}' 직원을 찾을 수 없습니다. 등록된 직원: {candidates}"

    except Exception as e:
        # 4) 예상 못한 예외도 그래프를 죽이지 않고 메시지로 변환
        return f"에러: 조회 중 문제가 발생했습니다 ({type(e).__name__}). 잠시 후 다시 시도하세요."


if __name__ == "__main__":
    print(search_employee.invoke({"name": "김"}))       # 너무 짧음
```

```
print(search_employee.invoke({"name": "홍길동"}))   # 없는 직원
print(search_employee.invoke({"name": "김하늘"}))   # 정상
```

2. 실행해서 세 가지 경우 모두 예외 없이 문자열이 돌아오는 것을 확인합니다. 이 도구를 Agent에 물리면 복구 흐름이 이렇게 됩니다.

```
사용자: "직원 김 정보 알려줘"
Agent -> search_employee(name="김")
도구  -> "에러: 이름이 너무 짧습니다. 최소 2자 이상..."
Agent -> (메시지를 읽고) 사용자에게 정확한 이름을 되묻거나
         맥락에서 이름을 추론해 다시 호출
```

막힐 때

- 에러 메시지를 "Error" 한 단어로만 돌려주면 LLM은 무엇을 고쳐야 할지 모릅니다. "무엇이 잘못됐고 다음에 어떻게 하라"까지 담아야 복구 루프가 돌아갑니다.
- 반대로 모든 예외를 삼키면 진짜 버그(코드 오타 등)를 놓칩니다. 개발 중에는 except 블록에서 `logging.exception(...)` 으로 스택트레이스를 남겨 둡니다. 그리고 반환값만 자연어로 바꿉니다.

확인: 세 가지 입력 모두 예외 없이 문자열로 결과가 나오고 실패 메시지에 다음 행동 안내가 담겨 있으면 성공입니다.

> 실패를 복구 지침이 담긴 메시지로 바꾸면 Agent가 스스로 복구한다는 원칙을 익혔습니다.
